Student Management System

Name: ***Aditya Kumar***

Technology: ***Java, Spring Boot, MongoDB, Postman***

STUDENT MANAGEMENT SYSTEM

1. ABSTRACT

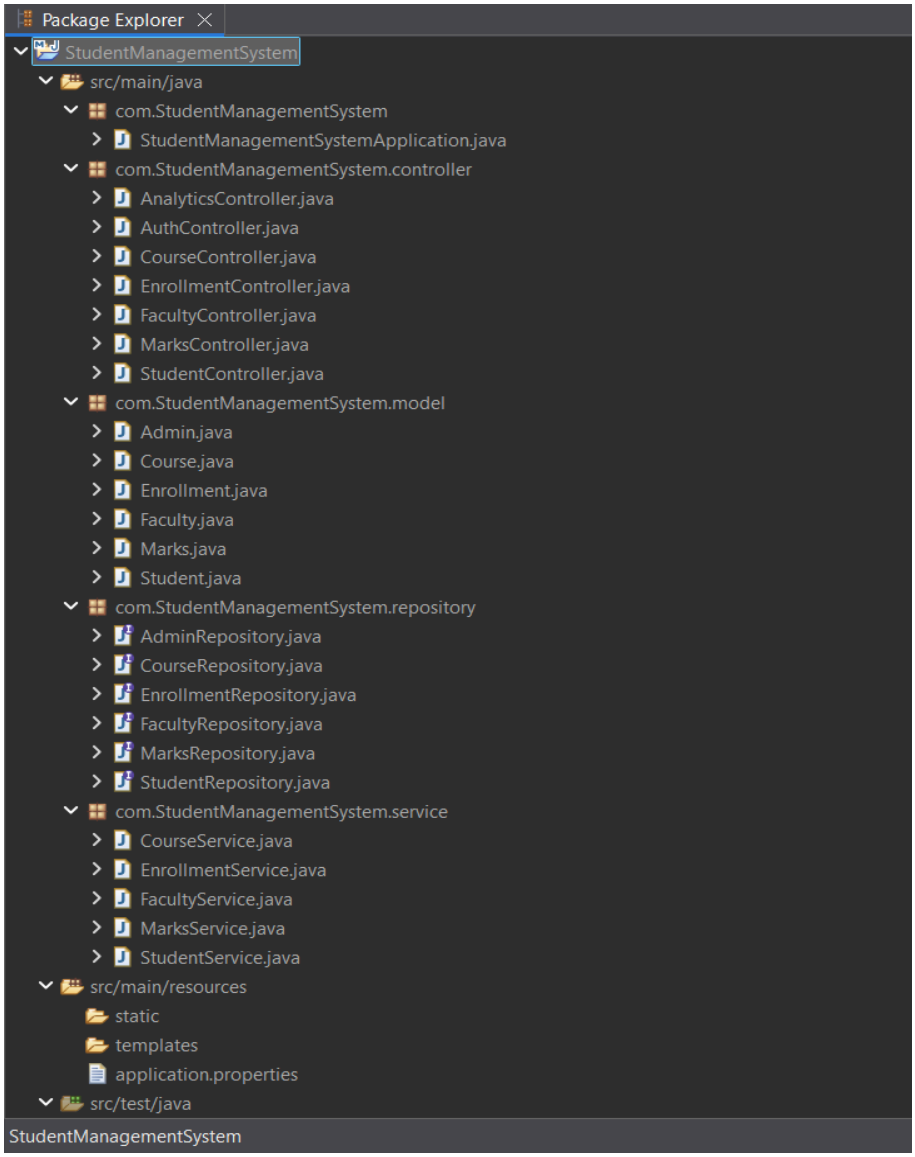
The Student Management System is a web-based application developed using Spring Boot and MongoDB. The system is designed to manage student-related information efficiently. It provides functionalities such as student registration, course management, faculty management, enrollment management, marks management, attendance management, and user authentication. The application exposes REST APIs that can be tested using Postman and stores data in MongoDB. The project demonstrates CRUD operations, database connectivity, and RESTful web service development.

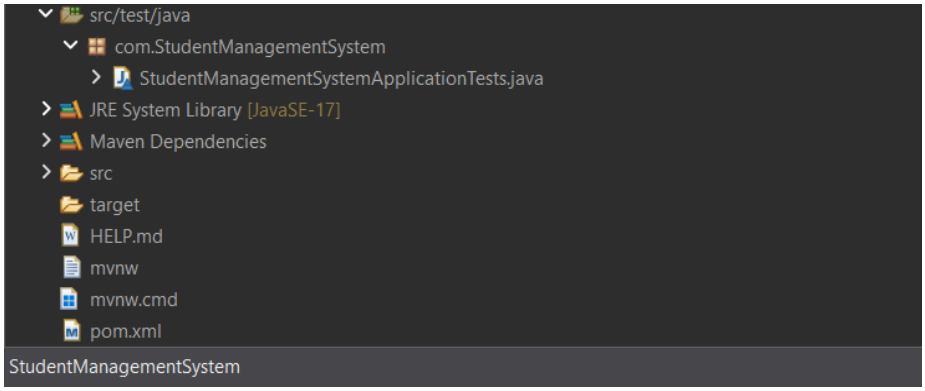
---

2. OBJECTIVES

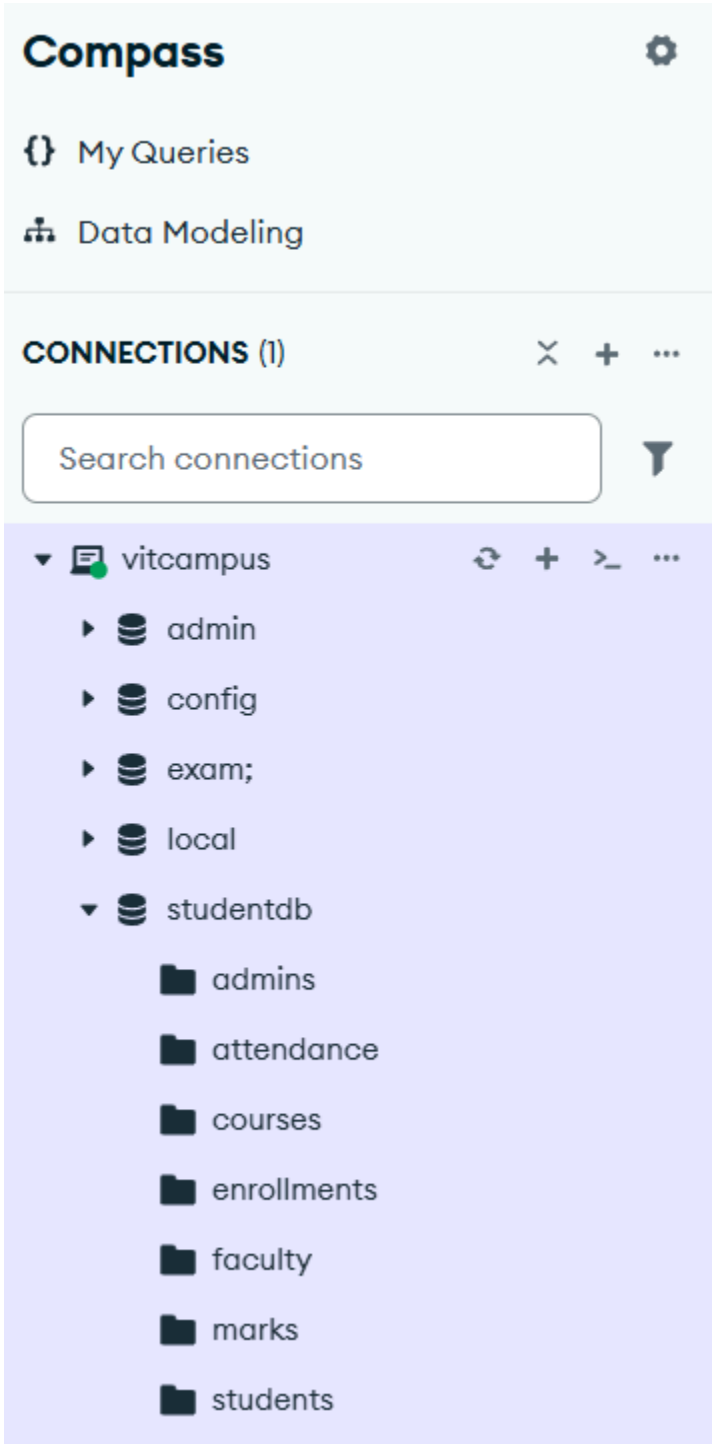
- To manage student records efficiently.
- To perform CRUD (Create, Read, Update, Delete) operations.
- To store and retrieve data using MongoDB.
- To implement REST APIs using Spring Boot.
- To manage courses, faculty, enrollments, marks, and attendance.
- To provide authentication for system access.
- To test APIs using Postman

Project Structure (Eclipse)





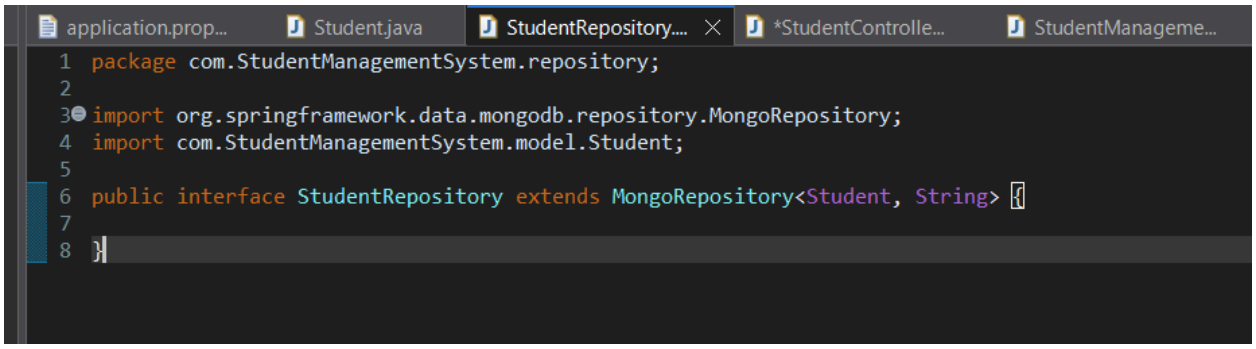
Mongodb


Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
vitcampus
admin
config
exam;
local
studentdb
admins
attendance
courses
enrollments
faculty
marks
students

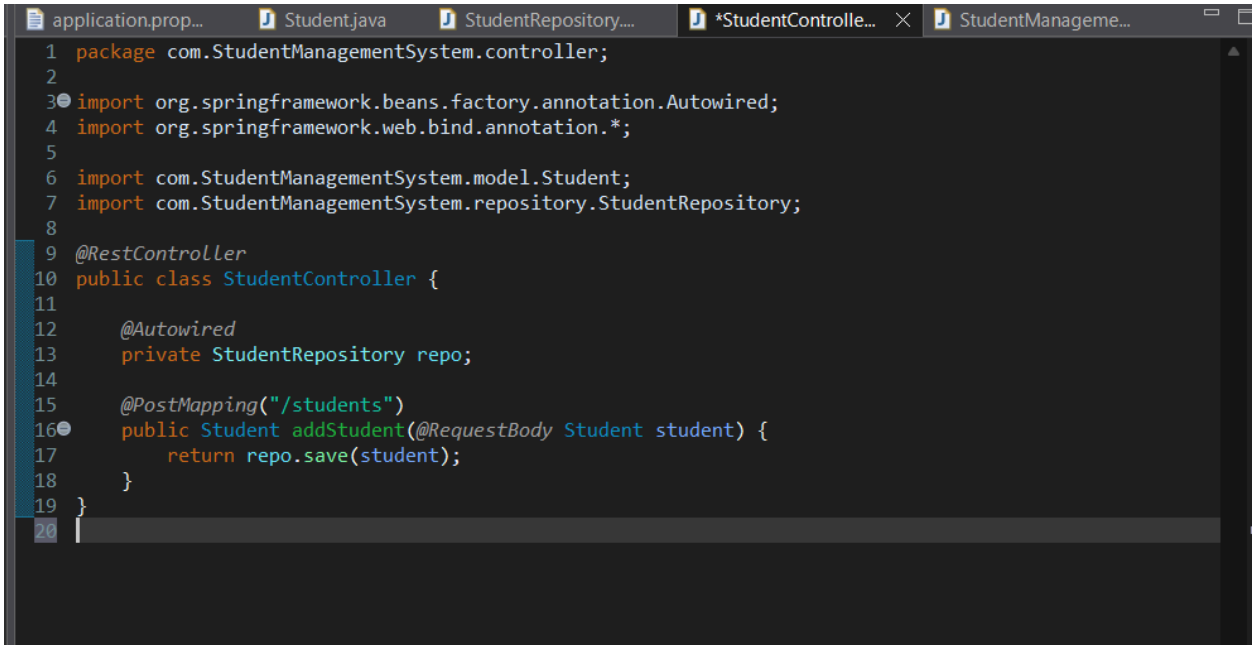
Student.java Code

```
45
46      public String getEmail() {
47          return email;
48      }
49
50      public void setEmail(String email) {
51          this.email = email;
52      }
53
54      public String getCourse() {
55          return course;
56      }
57
58      public void setCourse(String course) {
59          this.course = course;
60      }
61
62      public int getSemester() {
63          return semester;
64      }
65
66      public void setSemester(int semester) {
67          this.semester = semester;
68      }
69
70      public double getMarks() {
71          return marks;
72      }
73
74      public void setMarks(double marks) {
75          this.marks = marks;
76      }
77  }
```

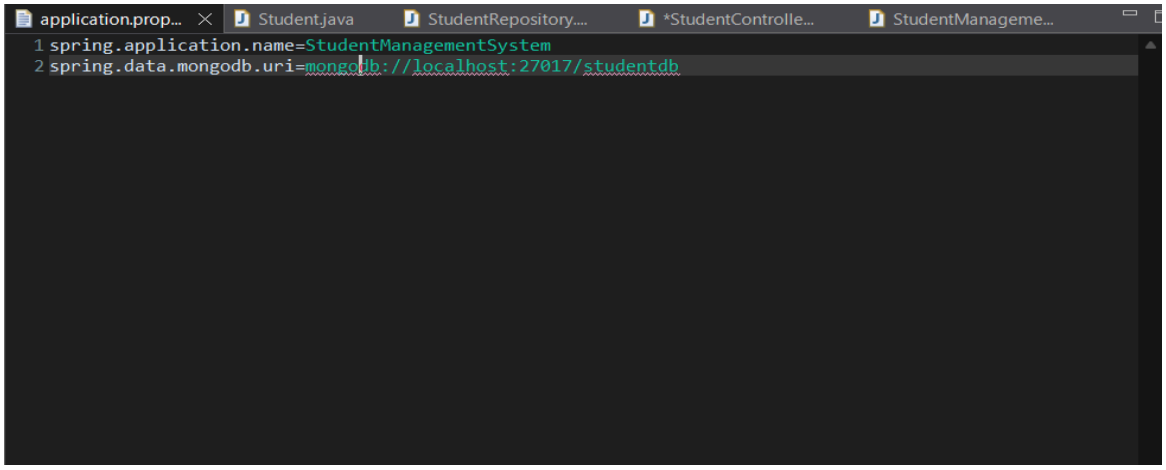
StudentRepository.java



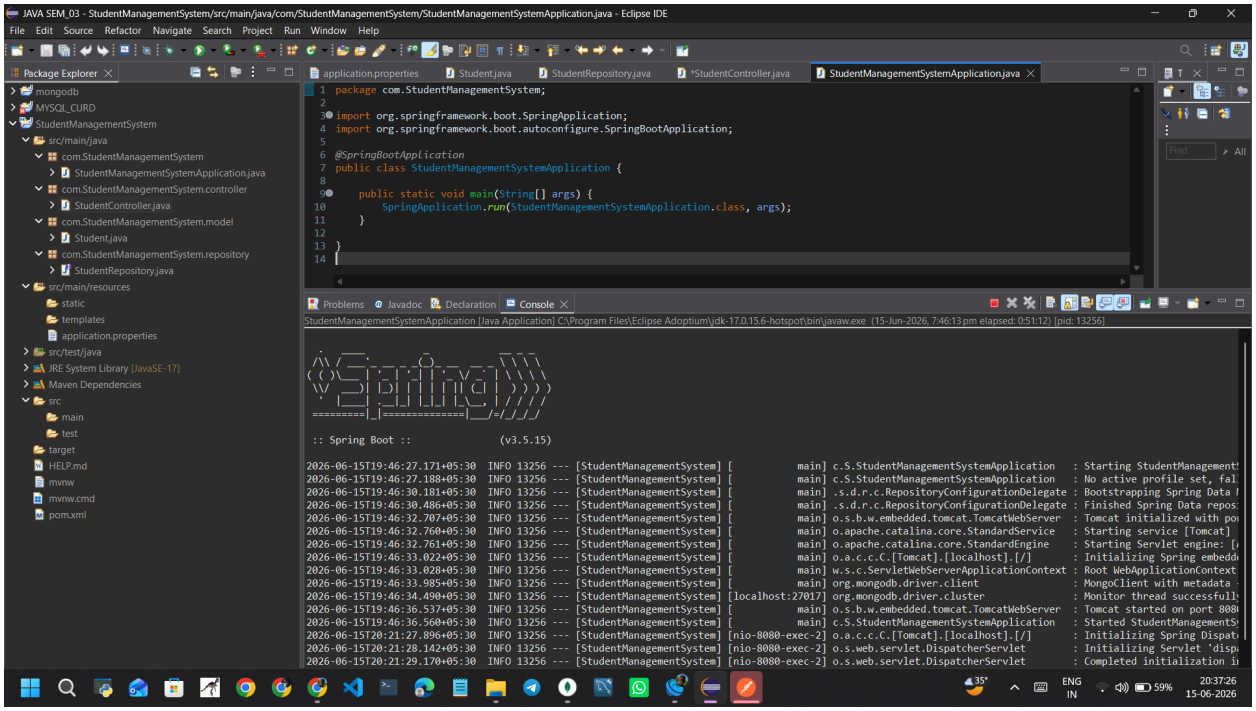
StudentController.java



Application.properties



Spring Boot Running



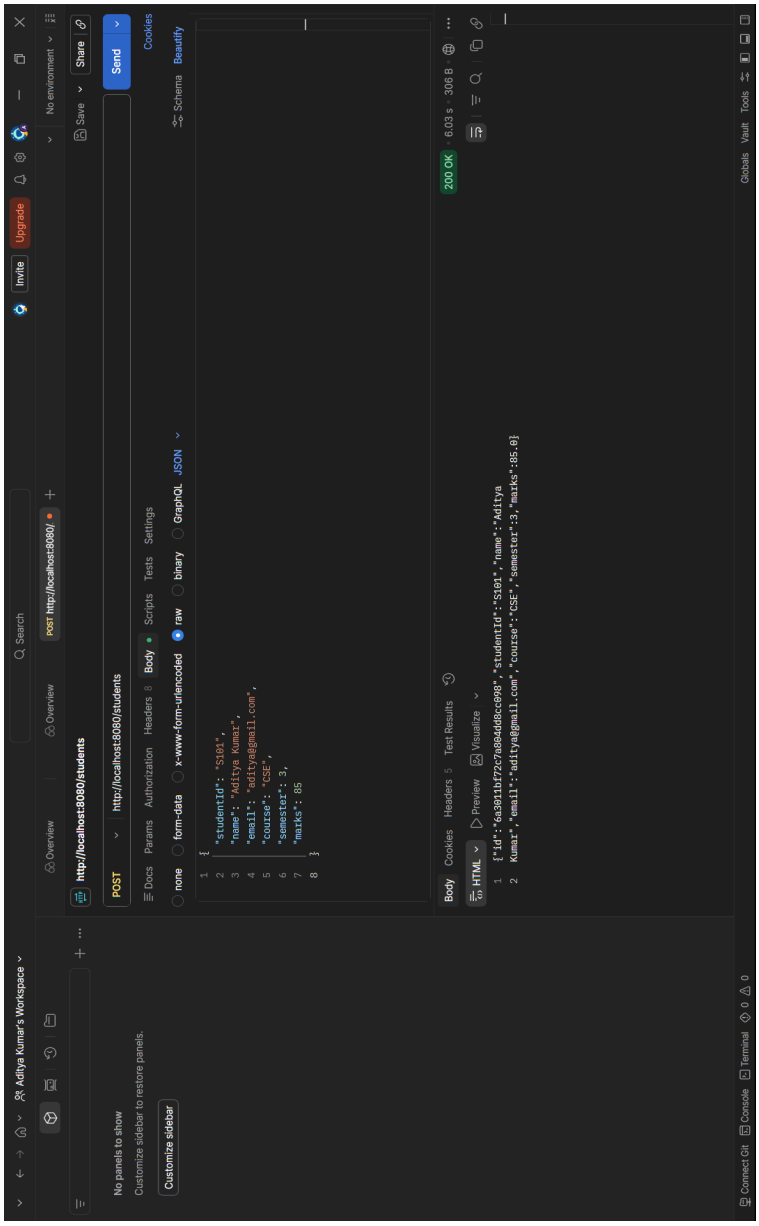
Postman Request and response

1. CREATE STUDENT

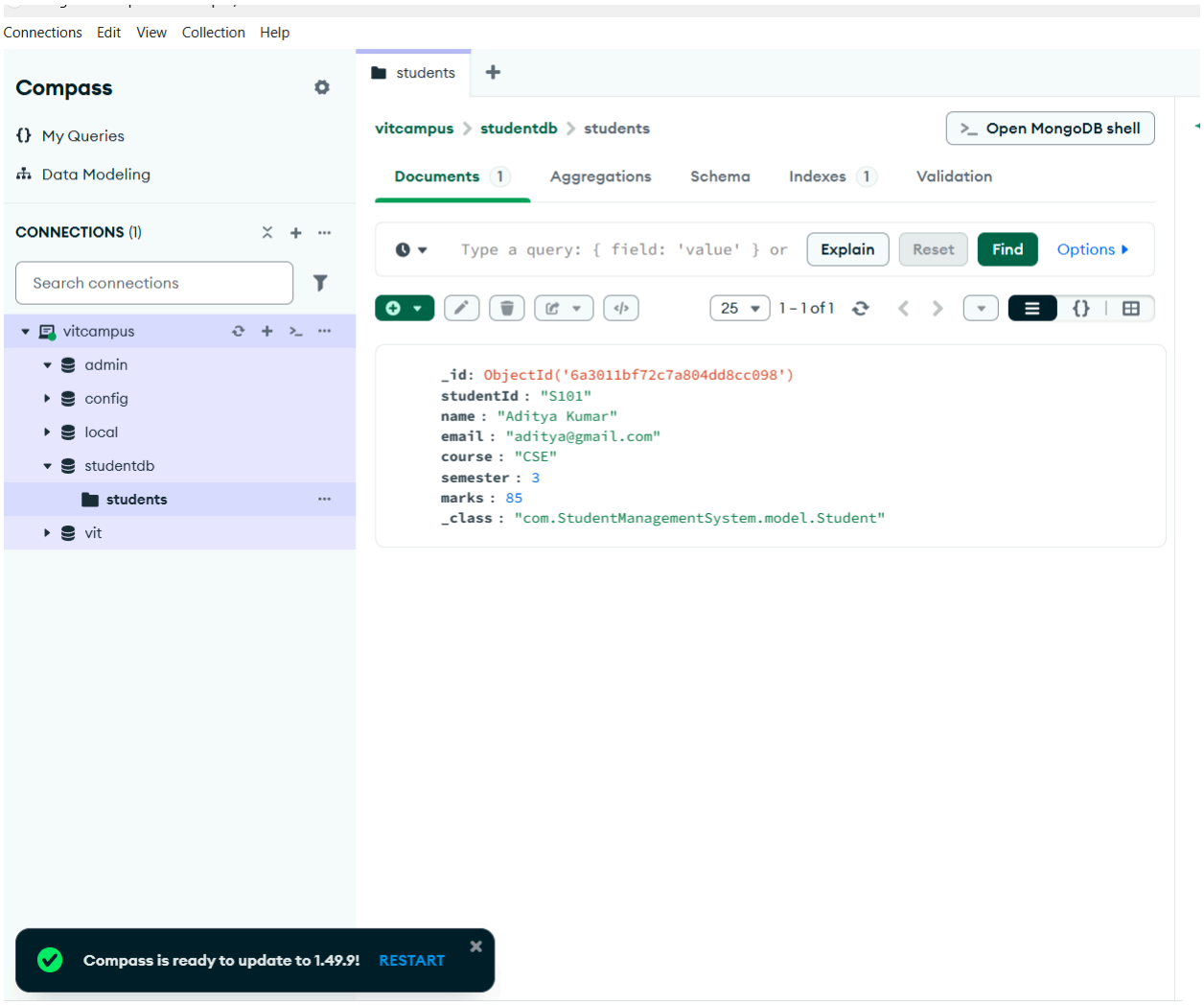
Method: POST

This API is used to add a new student record to the database. The user provides student details in JSON format, and the application stores the information in the MongoDB collection.

URL:
http://localhost:8080/students



MongoDB Compass



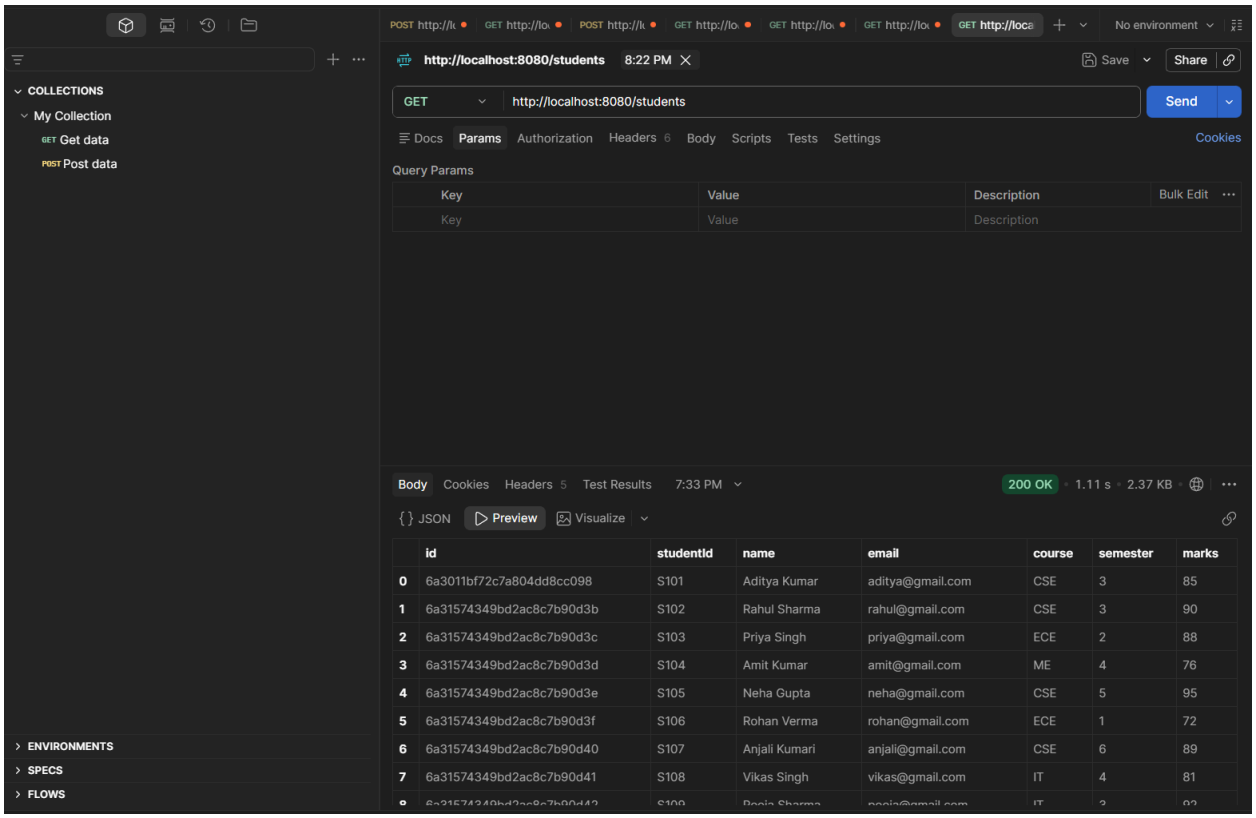
2. READ ALL STUDENTS

Method: GET

This API retrieves all student records available in the database. It helps administrators view and manage student information efficiently.

URL:
http://localhost:8080/students

Result:



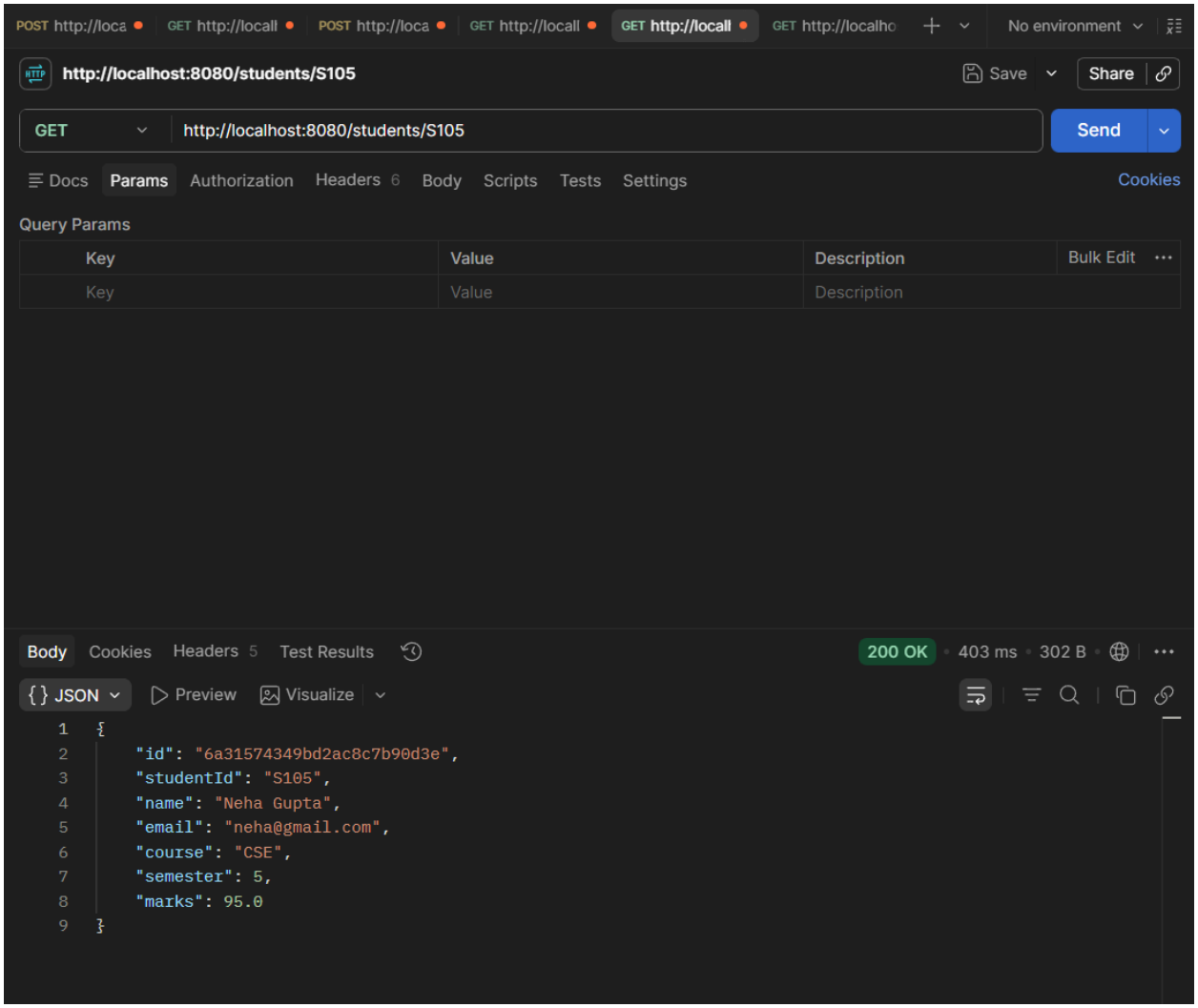
	id	studentId	name	email	course	semester	marks
0	6a3011bf72c7a804dd8cc098	S101	Aditya Kumar	aditya@gmail.com	CSE	3	85
1	6a31574349bd2ac8c7b90d3b	S102	Rahul Sharma	rahul@gmail.com	CSE	3	90
2	6a31574349bd2ac8c7b90d3c	S103	Priya Singh	priya@gmail.com	ECE	2	88
3	6a31574349bd2ac8c7b90d3d	S104	Amit Kumar	amit@gmail.com	ME	4	76
4	6a31574349bd2ac8c7b90d3e	S105	Neha Gupta	neha@gmail.com	CSE	5	95
5	6a31574349bd2ac8c7b90d3f	S106	Rohan Verma	rohan@gmail.com	ECE	1	72
6	6a31574349bd2ac8c7b90d40	S107	Anjali Kumari	anjali@gmail.com	CSE	6	89
7	6a31574349bd2ac8c7b90d41	S108	Vikas Singh	vikas@gmail.com	IT	4	81

3. READ STUDENT BY ID

Method: GET

URL:
http://localhost:8080/students/S105

Result:



.

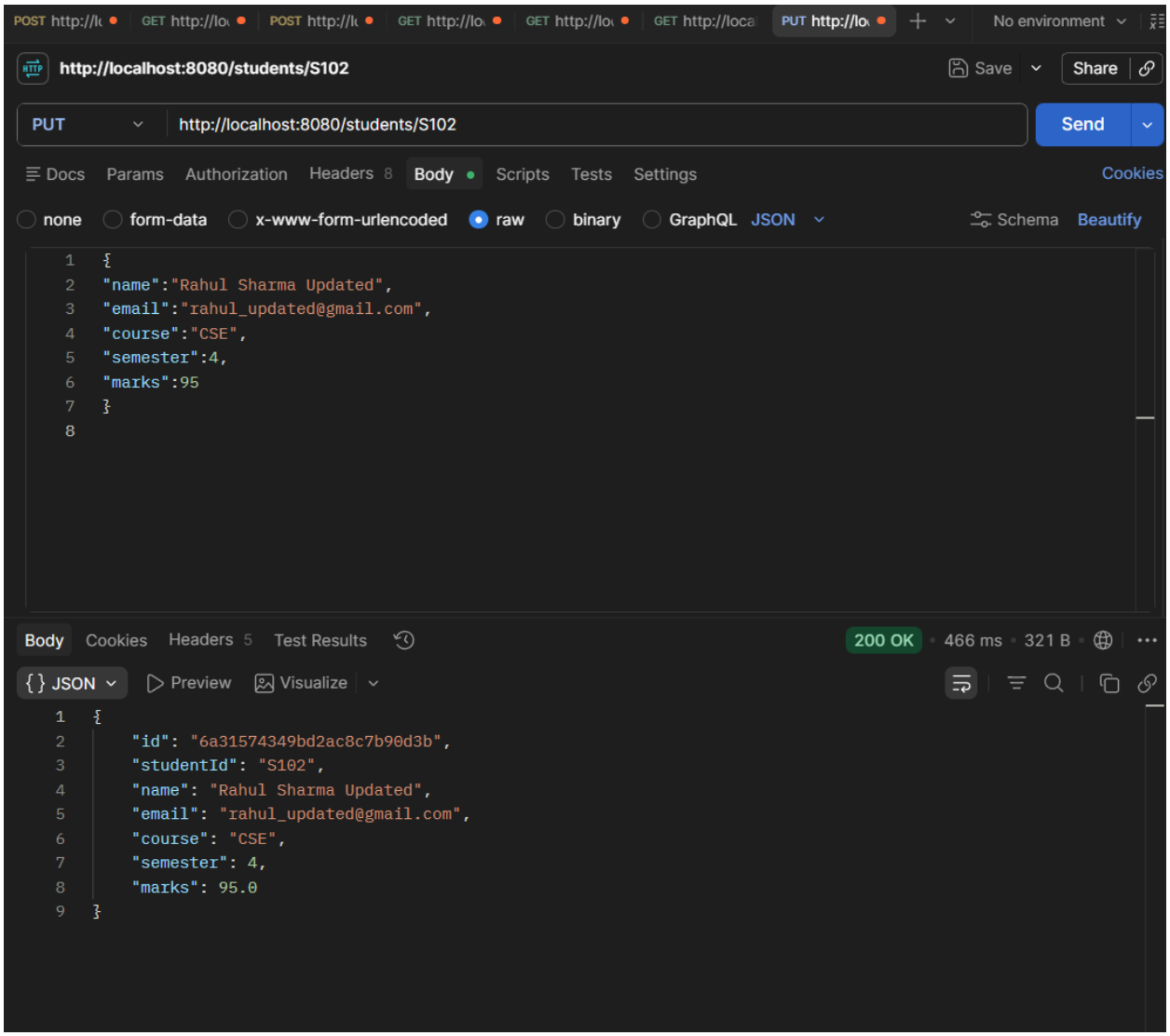
4. UPDATE STUDENT

Method: PUT

This API allows modification of an existing student record. Updated information is saved directly into the MongoDB database, ensuring data remains accurate and up to date.

URL:
http://localhost:8080/students/S102

Result:



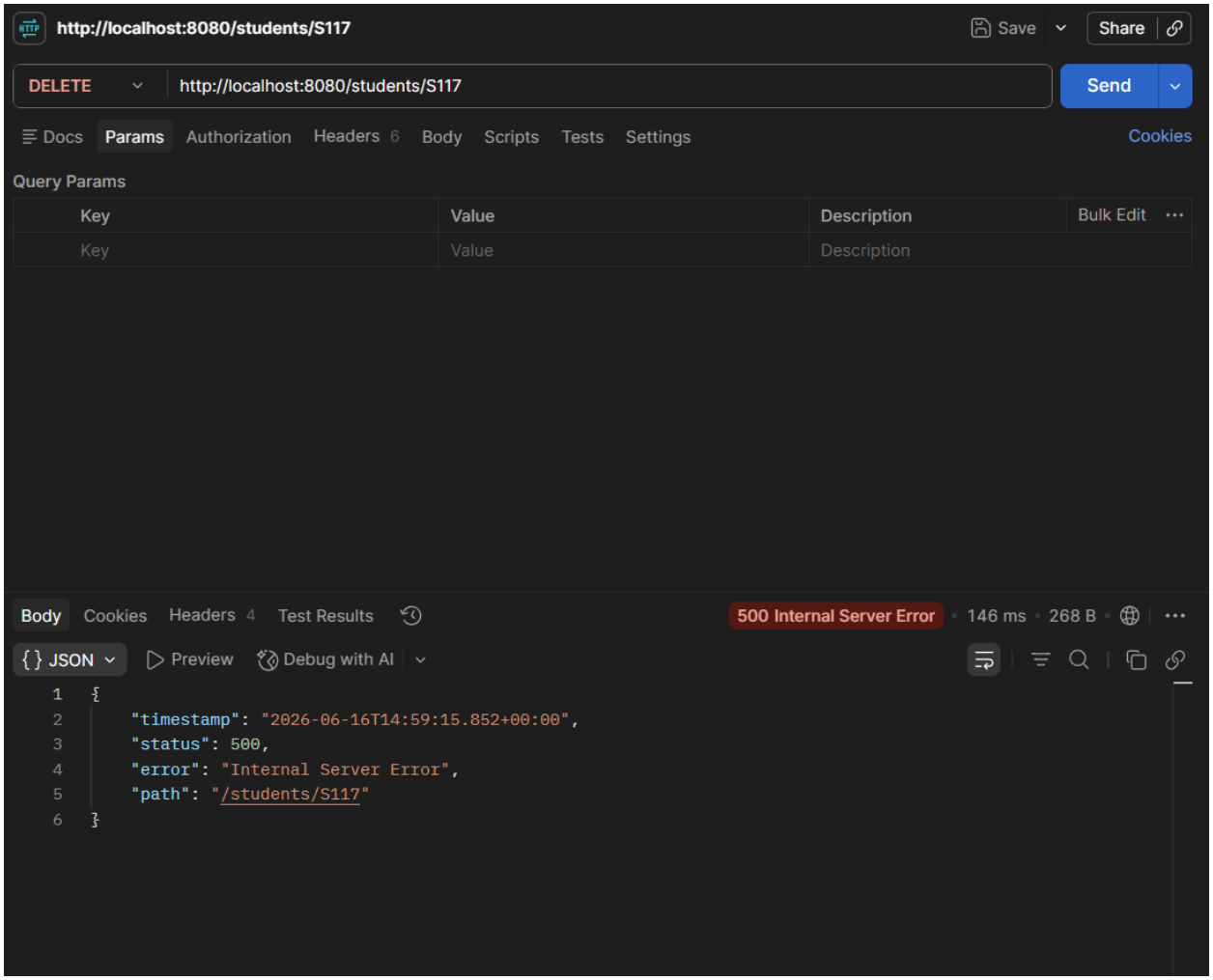
5. DELETE STUDENT

Method: DELETE

This API removes a student record from the database using the student's unique identifier. It demonstrates the delete operation of CRUD functionality.

URL:
http://localhost:8080/students/S117

Result:



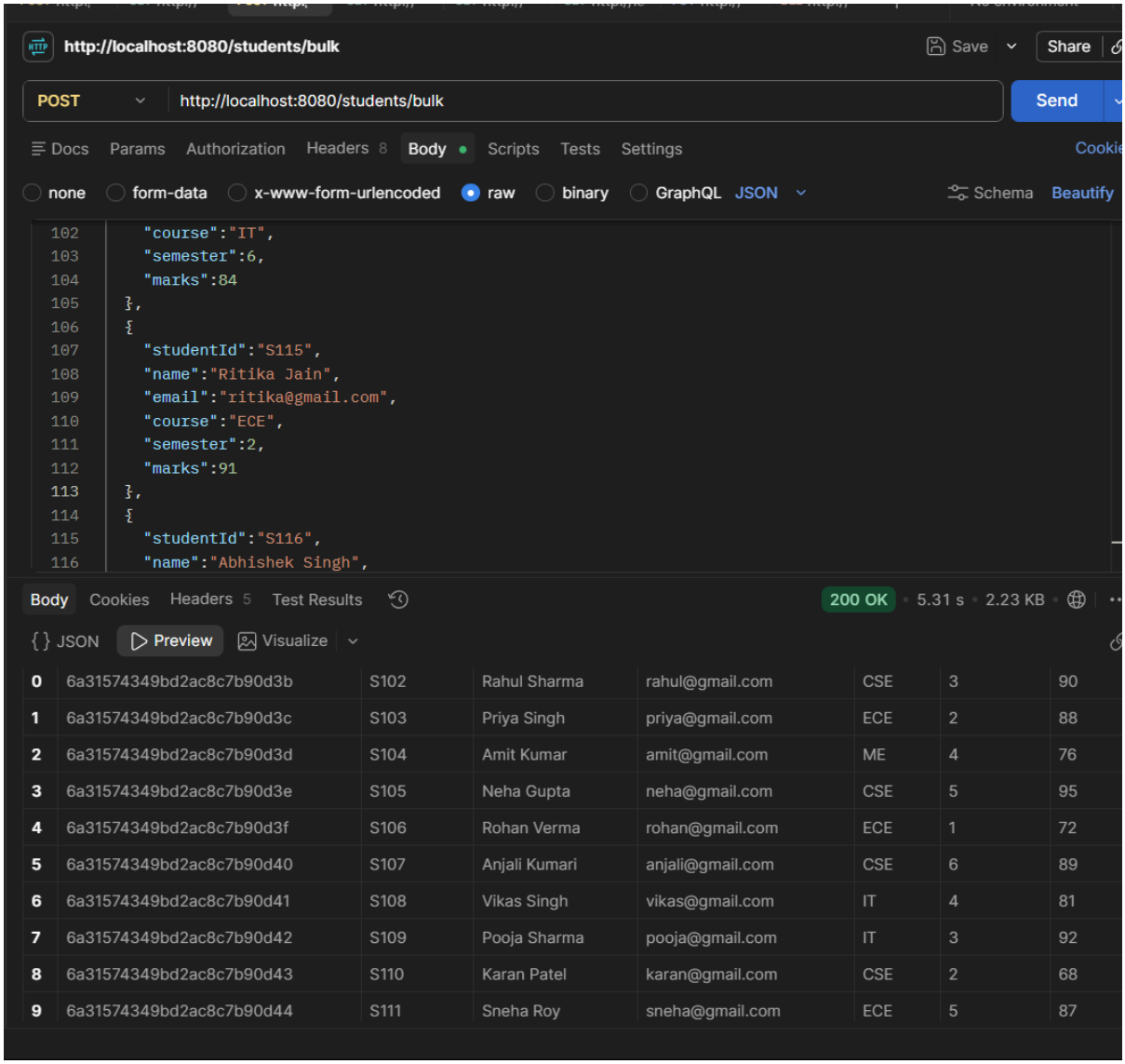
6. BULK INSERT STUDENTS

Method: POST

The bulk insert feature enables multiple student records to be added at once. This reduces manual effort and improves efficiency when handling large amounts of student data.

URL:
http://localhost:8080/students/bulk

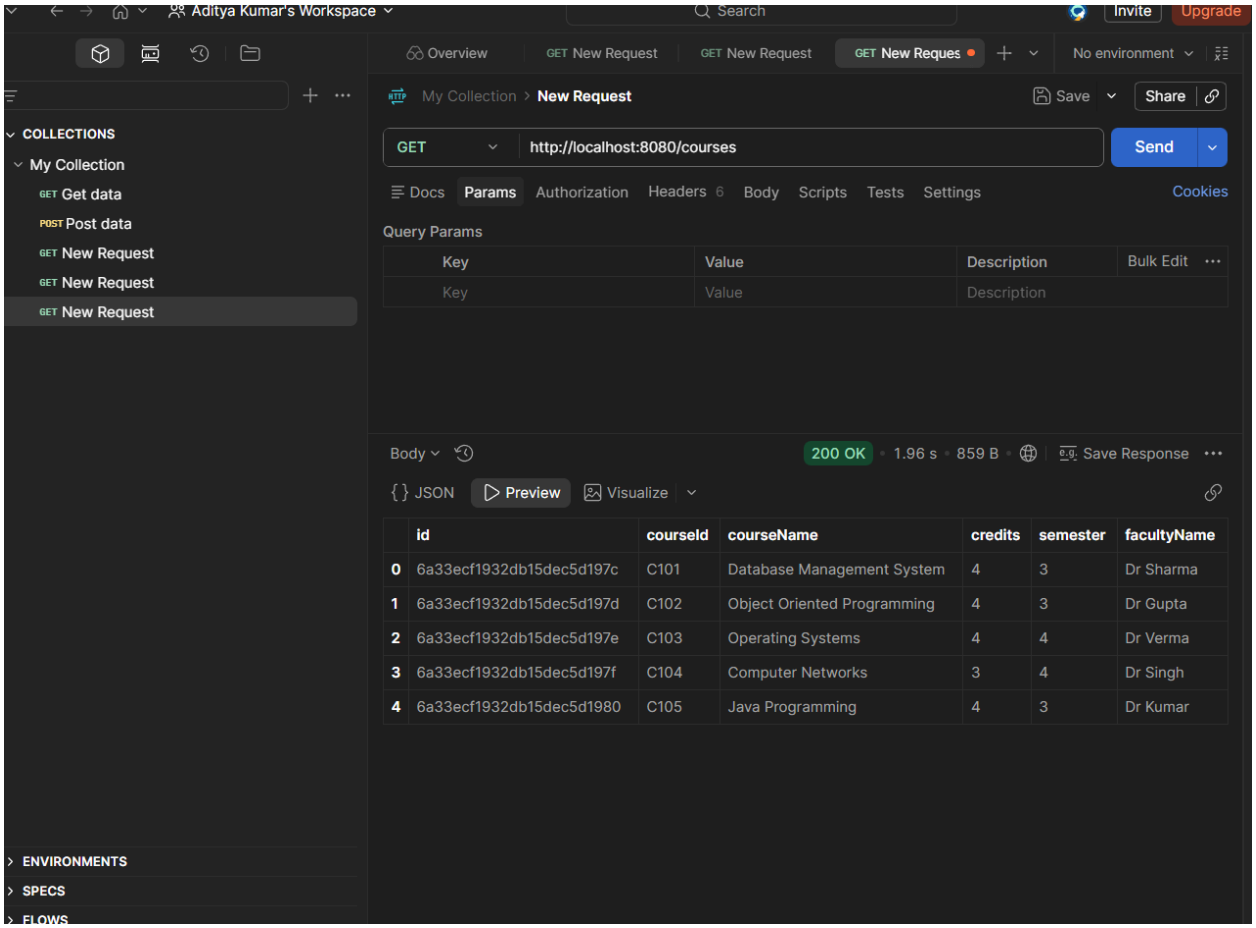
Result:
Multiple student records inserted into MongoDB at once.



Additional Module Screenshots

Course Module

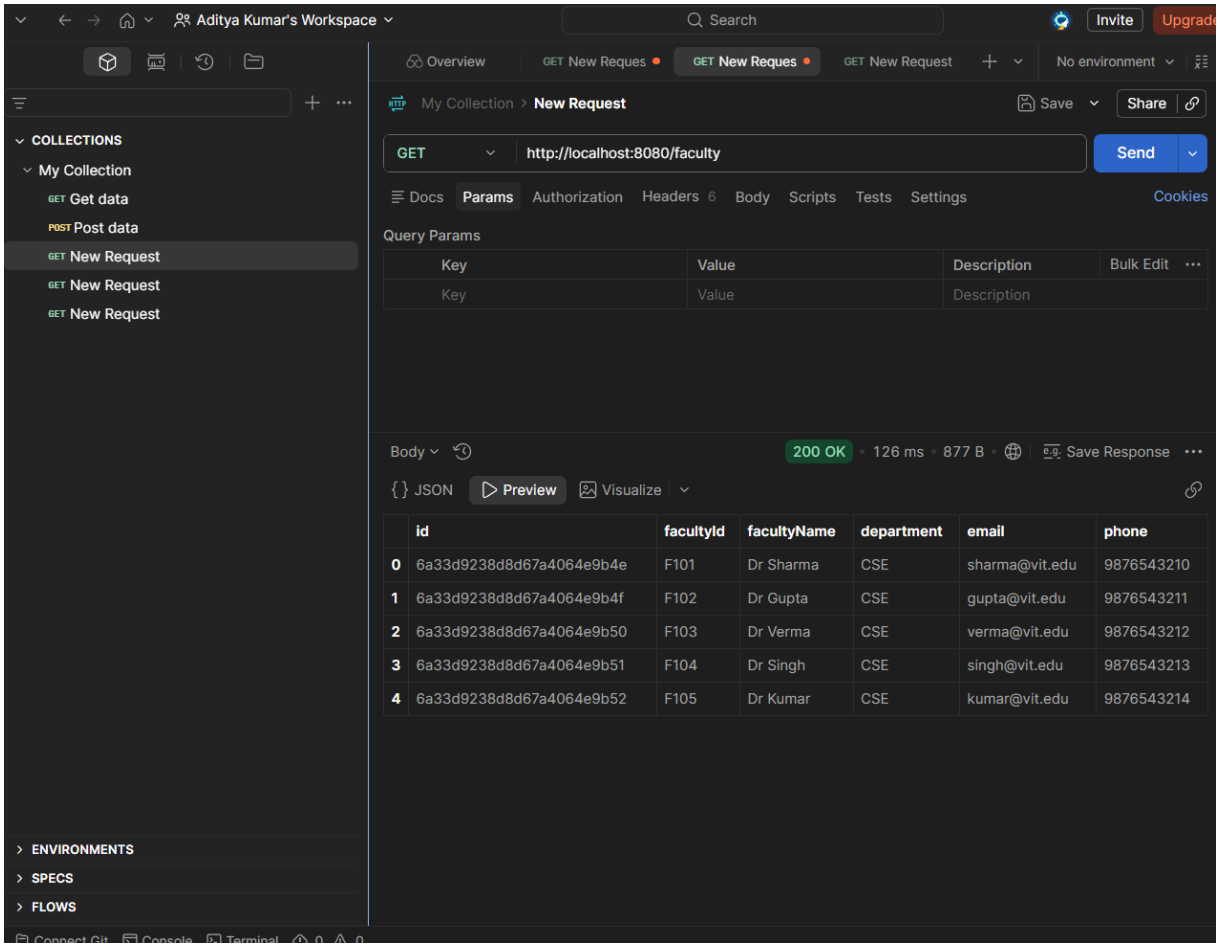
The Course Module stores and manages information about various courses offered by the institution. It helps maintain course details such as course ID, name, semester, and credits.



	id	courseId	courseName	credits	semester	facultyName
0	6a33ecf1932db15dec5d197c	C101	Database Management System	4	3	Dr Sharma
1	6a33ecf1932db15dec5d197d	C102	Object Oriented Programming	4	3	Dr Gupta
2	6a33ecf1932db15dec5d197e	C103	Operating Systems	4	4	Dr Verma
3	6a33ecf1932db15dec5d197f	C104	Computer Networks	3	4	Dr Singh
4	6a33ecf1932db15dec5d1980	C105	Java Programming	4	3	Dr Kumar

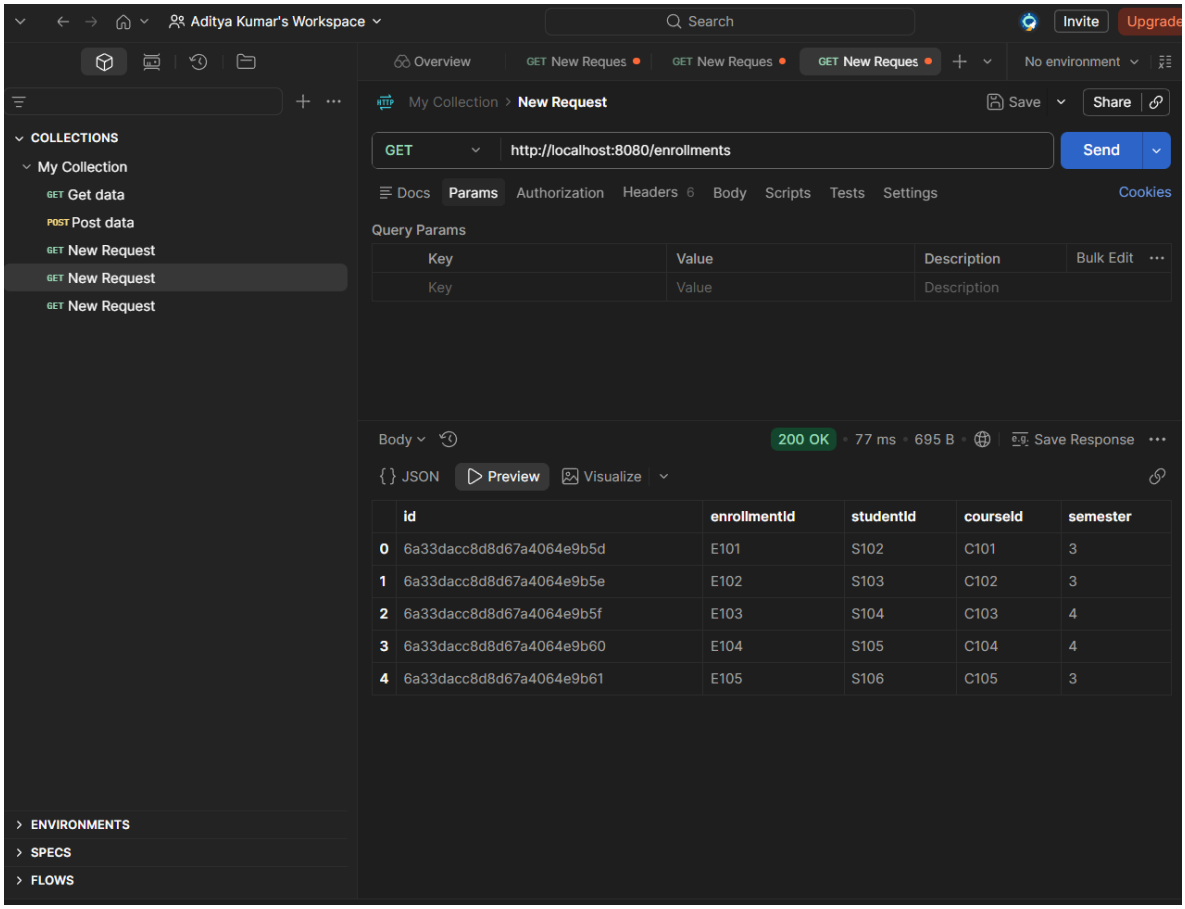
Faculty Module

The Faculty Module manages faculty-related information including faculty ID, name, department, email, and contact details. This helps maintain an organized faculty database.



Enrollment Module

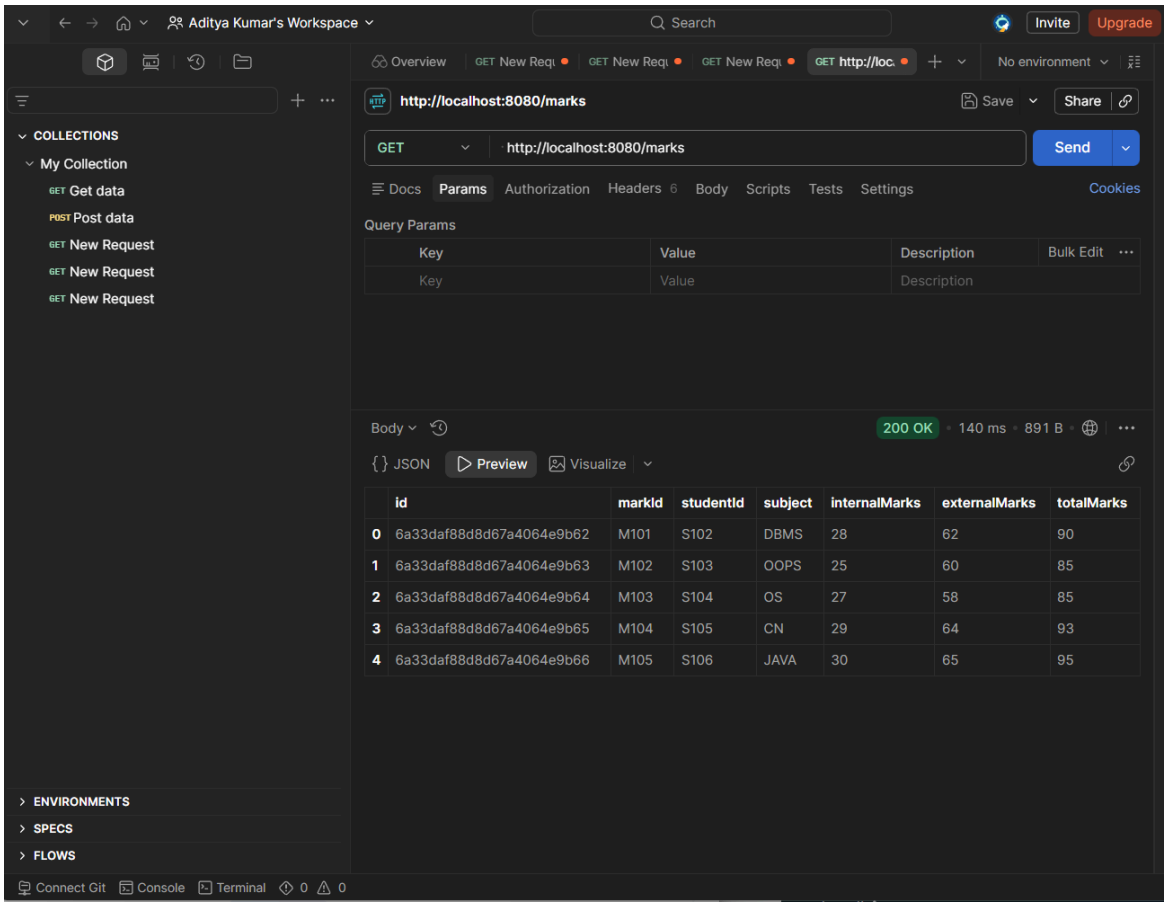
The Enrollment Module establishes the relationship between students and courses. It records which student is enrolled in which course and semester.



	id	enrollmentId	studentId	courseId	semester
0	6a33dacc8d8d67a4064e9b5d	E101	S102	C101	3
1	6a33dacc8d8d67a4064e9b5e	E102	S103	C102	3
2	6a33dacc8d8d67a4064e9b5f	E103	S104	C103	4
3	6a33dacc8d8d67a4064e9b60	E104	S105	C104	4
4	6a33dacc8d8d67a4064e9b61	E105	S106	C105	3

Marks Module

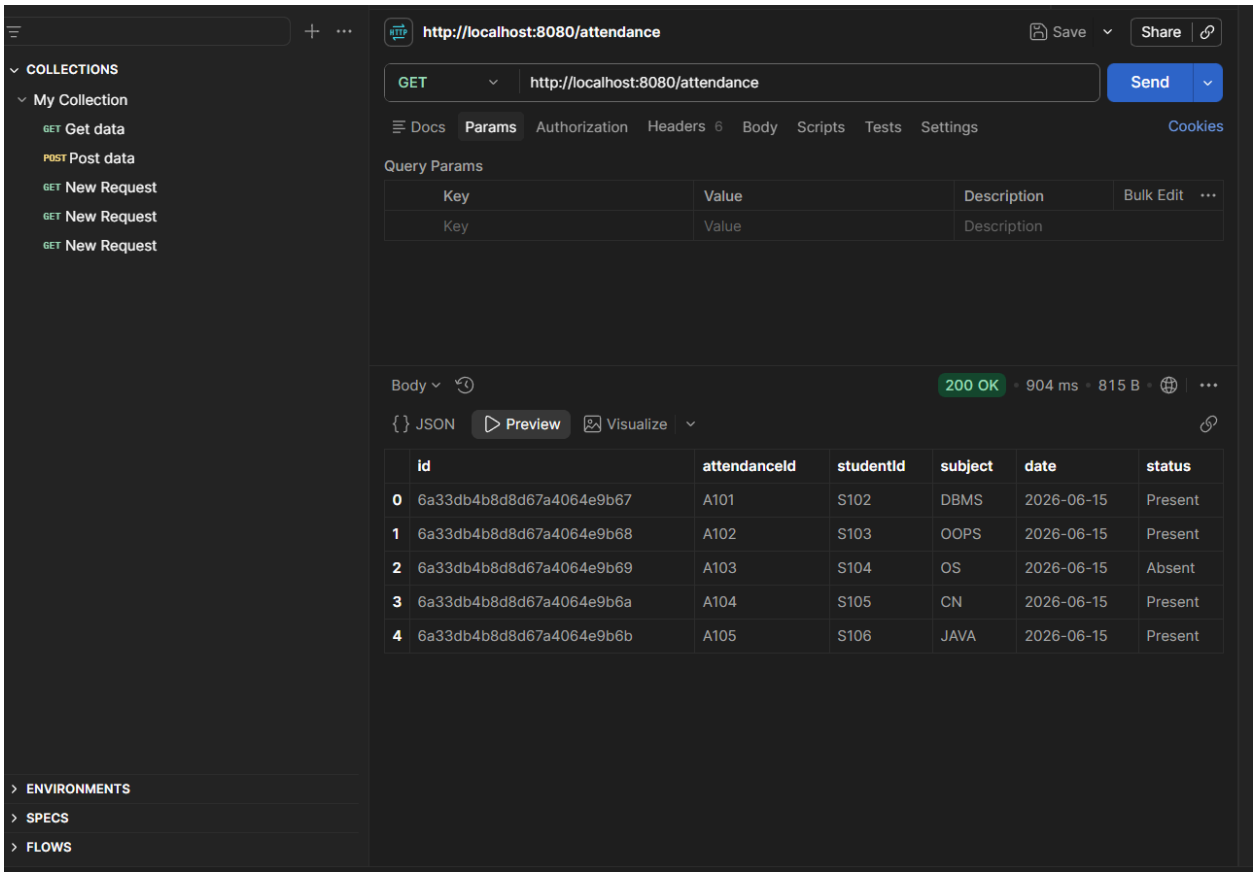
The Marks Module stores academic performance data of students. It records internal marks, external marks, and total marks for different subjects.



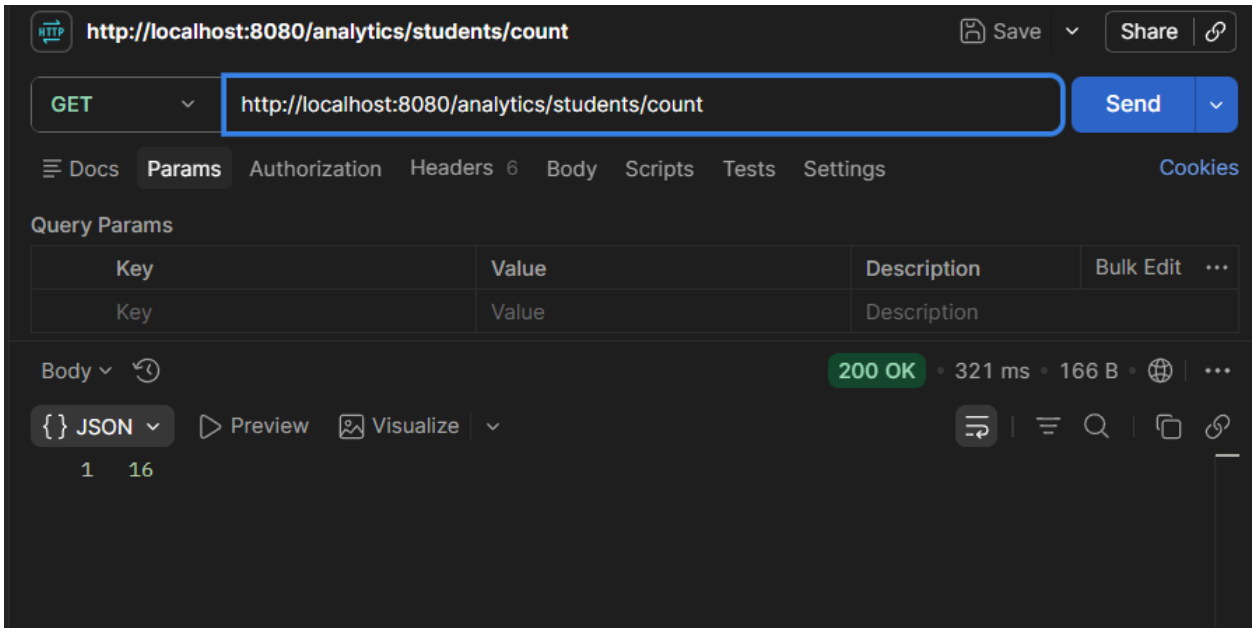
	id	markId	studentId	subject	internalMarks	externalMarks	totalMarks
0	6a33daf88d8d67a4064e9b62	M101	S102	DBMS	28	62	90
1	6a33daf88d8d67a4064e9b63	M102	S103	OOPS	25	60	85
2	6a33daf88d8d67a4064e9b64	M103	S104	OS	27	58	85
3	6a33daf88d8d67a4064e9b65	M104	S105	CN	29	64	93
4	6a33daf88d8d67a4064e9b66	M105	S106	JAVA	30	65	95

Attendance Module

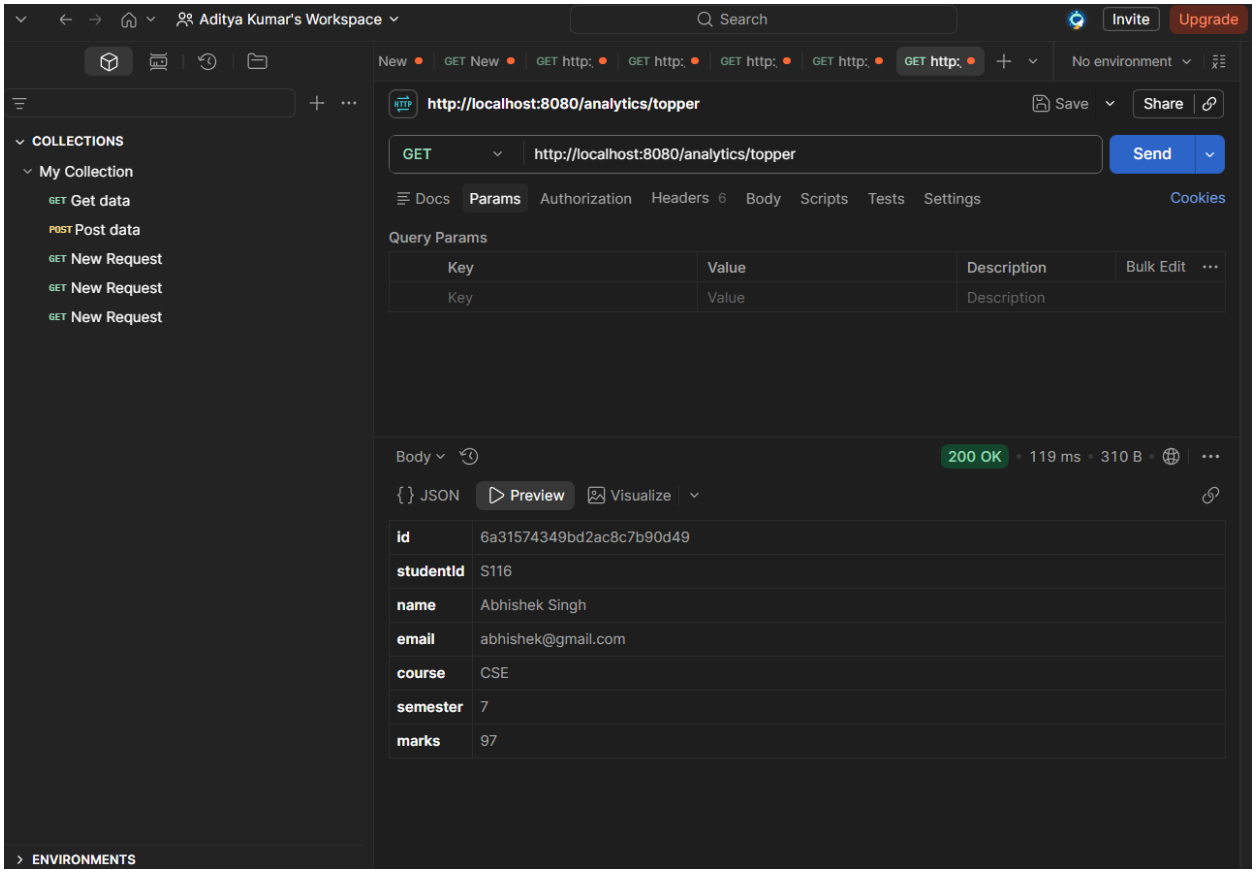
The Attendance Module maintains attendance records for students. It tracks attendance status, subject information, and attendance dates for academic monitoring.



Analytics



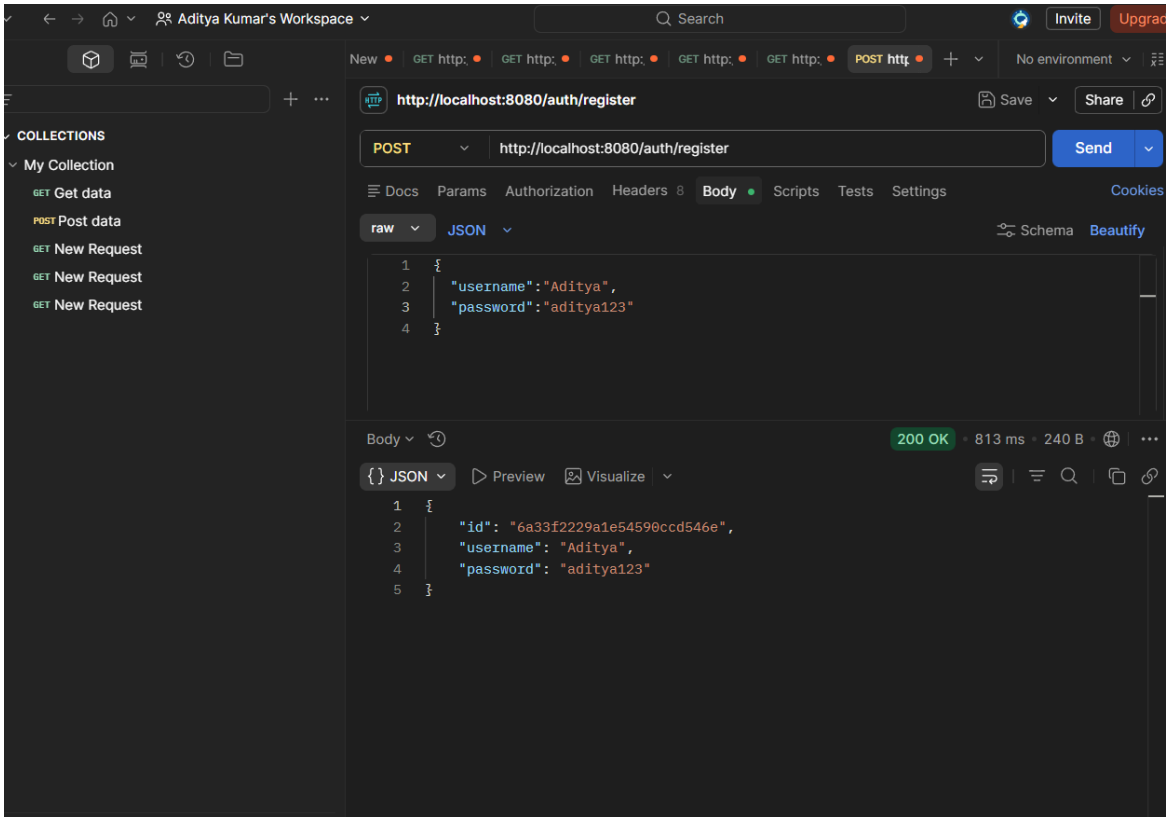
topper?



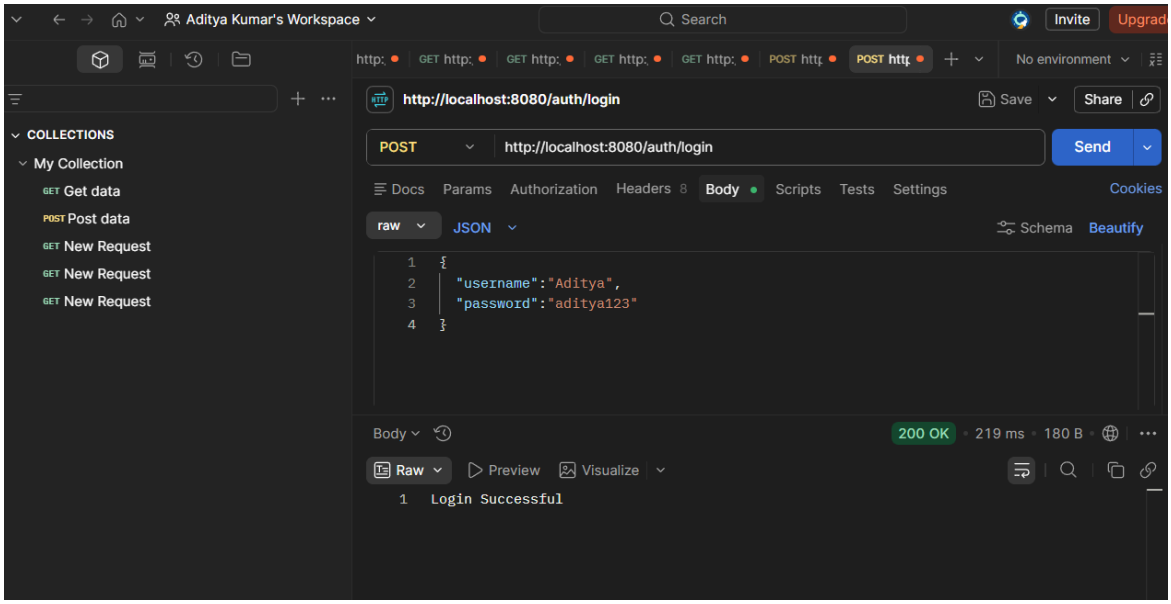
Authentication

The Authentication Module provides a basic login mechanism for system access. User credentials are verified before granting access, enhancing application security.

Register



Login


Aditya Kumar's Workspace
Search
Invite
Upgrad
No environment
COLLECTIONS
My Collection
GET Get data
POST Post data
GET New Request
GET New Request
GET New Request
http://localhost:8080/auth/login
Save
Share
POST
http://localhost:8080/auth/login
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Tests
Settings
Cookies
raw
JSON
Schema
Beautify
{
"username":"Aditya",
"password":"aditya123"
}
Body
200 OK
219 ms
180 B
Raw
Preview
Visualize
Login Successful

Conclusion

The Student Management System was successfully developed using Spring Boot and MongoDB. The project demonstrates CRUD operations, REST API development, database connectivity, authentication, and data management features. The system provides an efficient way to manage students, courses, faculty, enrollments, marks, and attendance records. This project helped in understanding backend development concepts and NoSQL database integration in real-world applications.